AI Research Copilot

Week 1 Complete — Your Personal Field Manual

What You Built in Week 1

You went from zero to a live, publicly accessible AI Research Copilot in 7 days. Here is everything you built, every mistake you made, and every concept you need to understand deeply.

- Live Backend API on Railway: ai-copilot-production-c2eb.up.railway.app
- Live Frontend on Vercel: ai-copilot-sand.vercel.app
- 4 working endpoints: /research, /compare, /summarize, /trending
- Real web search via Tavily + LLM summarization via Groq

Phase 0 — Your Starting Stack (What Was Already Ready)

Before Week 1 started, you had already completed Phase 0. This was your foundation:

- Cursor — your code editor
- Python 3.11 + Node 22 LTS — language runtimes
- Git + GitHub — version control
- Groq API — LLM provider (model: llama-3.3-70b-versatile)
- Railway — backend deployment platform
- Supabase — database (not used yet, coming later)
- Vercel — frontend deployment platform
- First FastAPI app already live on Railway

Day 1 — The /research Endpoint

What you built

An endpoint that takes a topic, sends it to Groq with a structured prompt, and returns a clean JSON response.

Key concept: Structured Output via Prompting

Instead of asking the LLM to 'tell me about X', you told it to respond ONLY in a specific JSON format. This is prompt engineering — forcing the model to give you machine-readable output.

```
prompt = f"""Respond ONLY with a JSON object:
{{
  "topic": "{topic}",
  "summary": "...",
  "key_points": [...]
}}"""
```

Why json.loads(raw)?

The LLM returns a string. json.loads() converts that string into a real Python dictionary, which FastAPI then returns as proper JSON. Without this, you'd return raw text, not JSON.

What you deployed

After testing locally at localhost:8000/research?topic=quantum computing, you pushed to Railway via git push and confirmed it live on the Railway URL.

Day 2 — Adding Real Web Search (Tavily)

The problem with Day 1

Your /research endpoint only used Llama's training data — which has a knowledge cutoff. It couldn't tell you about current events or recent news.

The fix: Tavily API

Tavily is a search API built specifically for LLM apps. You sign up, get an API key, and call it before your LLM prompt. It returns real web results which you then pass as context to Groq.

```
results = tavily.search(query=topic, max_results=3)
context = "\n".join([r["content"] for r in results["results"]])
```

The mistake you made

■ After installing tavily-python locally with pip install, you pushed to Railway and it crashed. Error: ModuleNotFoundError: No module named tavily

Root cause: Railway installs from requirements.txt, not your local venv. Fix:

```
pip freeze > requirements.txt
git add .
```

```
git commit -m "update requirements"
git push
```

✓ Always run pip freeze > requirements.txt after installing any new package before pushing.

Environment Variables

You added TAVILY_API_KEY to Railway's Variables section. Railway injects these as environment variables at runtime — same as your .env file locally. Your code reads both the same way: os.getenv('TAVILY_API_KEY').

Day 3 — The /compare Endpoint

What you built

An endpoint that takes two topics, searches both via Tavily, and returns a structured comparison with similarities, differences, and a verdict.

Key concept: Multiple API calls in one endpoint

You made two separate Tavily searches (one per topic) and combined both results into a single prompt for Groq. This is a pattern you'll use constantly — gather context from multiple sources, then summarize with LLM.

```
results_a = tavily.search(query=topic_a, max_results=2)
results_b = tavily.search(query=topic_b, max_results=2)
```

Query parameters with multiple inputs

Your endpoint uses two query params: topic_a and topic_b. You test it like this:

```
localhost:8000/compare?topic_a=python&topic;_b=javascript
```

Day 4 — The /summarize Endpoint

What you built

An endpoint that takes any URL, fetches the full article content, and returns a structured summary using Groq.

New library: newspaper3k

This library downloads and parses web articles. Two key steps: article.download() fetches the HTML, article.parse() extracts clean text and title.

```
article = Article(url)
article.download()
article.parse()
# Now use: article.title and article.text
```

Why [:3000]?

LLMs have token limits. Passing the entire article could exceed the context window or cost more tokens. Slicing to 3000 characters gives enough content for a good summary without overloading the prompt.

Installation note

You installed both newspaper3k and lxml_html_clean — the second one is a dependency that newspaper3k needs but doesn't always install automatically.

Day 5 — Frontend UI + CORS Fix

What you built

A single index.html file with cards for each endpoint — Research, Compare, Summarize, Trending. Pure HTML + vanilla JavaScript, no frameworks.

The CORS error — most important mistake of Week 1

■ Access to fetch blocked by CORS policy: No Access-Control-Allow-Origin header

CORS (Cross-Origin Resource Sharing) is a browser security rule. When your HTML (on one origin) tries to call your Railway API (different origin), the browser blocks it unless the API explicitly allows it.

Fix — add this middleware to main.py:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

■ allow_credentials=True does NOT work with allow_origins=['*'] — it throws an error. Never combine these two.

Vercel deployment

You deployed index.html to Vercel as a static site. Key understanding: Vercel hosts your frontend (HTML/CSS/JS), Railway hosts your backend (Python API). They work together — the HTML calls the Railway API via fetch().

Day 6 — The /trending Endpoint + Vercel/GitHub Connection

What you built

An endpoint that takes a category (AI, crypto, tech) and returns current trending topics with summaries, powered by a real-time Tavily search.

The ID mismatch bug

■ TypeError: Cannot read properties of null (reading 'value') at trending

Your HTML input had id='trendingInput' but your JavaScript was looking for id='trendingCategory'. JavaScript returned null because the element didn't exist. Always make sure HTML ids match exactly what JS is looking for.

Connecting Vercel to GitHub

You connected your Vercel project to your GitHub repo so every git push auto-deploys. The empty commit trick to force a redeploy:

```
git commit --allow-empty -m "trigger deploy"
git push
```

✓ You'll never need this again — every real code change will trigger auto-deploy now.

Day 7 — UI Polish

What you added

- Loading spinner on buttons while API is processing
- Buttons disabled during loading — prevents double clicks
- Error handling — shows red error message if API fails
- Better visual design — borders, letter-spacing, transitions

Key pattern: reusable callAPI function

Instead of repeating fetch logic in every function, you created one callAPI() function that handles loading state, error handling, and output display. This is DRY principle — Don't Repeat Yourself.

Critical Concepts to Remember

1. Virtual Environment

Always activate .venv before running anything: .venv\Scripts\activate. If you see 'module not found' locally, your venv is not activated.

2. requirements.txt is your contract with Railway

Railway has no idea what you installed locally. It only installs what's in requirements.txt. Run pip freeze > requirements.txt every time you install something new.

3. Environment Variables

Never hardcode API keys. Store them in .env locally (loaded via dotenv) and in Railway/Vercel Variables for production. Your code uses os.getenv() to read them in both environments.

4. Structured Prompting = Reliable Output

Telling the LLM to respond 'ONLY with a JSON object in this exact format, nothing else' is what makes json.loads() work reliably. Without the strict prompt, the LLM might add extra text that breaks JSON parsing.

5. CORS is a browser rule, not an API rule

Your Railway API works fine from Postman or browser URL bar. CORS only kicks in when JavaScript in a browser tries to fetch from a different domain. The fix lives in your FastAPI backend, not the frontend.

Your Complete main.py After Week 1

Here is what your final main.py looks like after all 7 days:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from groq import Groq
from tavily import TavilyClient
from newspaper import Article
```

```
from dotenv import load_dotenv
import os, json

load_dotenv()
app = FastAPI()
client = Groq(api_key=os.getenv("GROQ_API_KEY"))
tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

app.add_middleware(CORSMiddleware, allow_origins=["*"],
    allow_methods=["*"], allow_headers=["*"])

@app.get("/")
def root(): return {"status": "alive"}

@app.get("/ask")
def ask(query: str): ...

@app.get("/research")
def research(topic: str): ...

@app.get("/compare")
def compare(topic_a: str, topic_b: str): ...

@app.get("/summarize")
def summarize(url: str): ...

@app.get("/trending")
def trending(category: str): ...
```

Week 2 Preview

Here is what is coming in Week 2:

- Day 8 — Conversation history (multi-turn memory)
- Day 9 — /fact-check endpoint
- Day 10 — Save research results
- Day 11 — Proper chat UI
- Day 12 — Performance improvements
- Day 13-14 — Polish + public ship

You are no longer a beginner. You have a live product. Week 2 makes it smarter.

— End of Week 1 Report —